

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللهی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 7/30

More on Structured programming, working with an unknown number of data, and more

- Selection statements: the switch statement
- Working with an unknown number of data
- Infinite loops
- References and further readings
- Q&A

120 min (incl. Q & A)



Selection statements: the **S**witch statement

Selection statements: the **S**witch statement

- Selection statement

- if statement

if (condition)
statement

if (condition)
statement₁
else
statement₂

- switch statement

switch (condition)
statement

- The switch statement causes control to be transferred to one of several statements depending on the value of a condition.
- A **switch**-statement tests a value against a set of constants. Those constants, called **case**-labels, must be distinct, and if the value tested does not match any of them, the **default** is chosen. If the value doesn't match any case-label and no default is provided, no action is taken.

Selection statements: if vs. Switch statement

```
bool accept()
{
    cout << "Do you want to proceed (y or n )? "; // write question
    char answer;
    cin >> answer; // read answer
    if (answer == 'y')
        return true;
    return false;
}
```

```
bool accept2()
{
    cout << "Do you want to proceed (y or n)?\n"; // write question
    char answer = 0; // initialize to a value that will not appear on input
    cin >> answer; // read answer
    switch (answer) {
        case 'y':
            return true;
        case 'n':
            return false;
        default:
            cout << "I'll take that for a no.\n";
            return false;
    }
}
```

switch statement and Break

```
bool accept3()
{
    cout << "Do you want to proceed (y or n)?\n"; // write question
    char answer = 0; // initialize to a value that will not appear on input
    cin >> answer; // read answer
    bool result;
    switch (answer) {
        case 'y':
        case 'Y':
            result = true;
            break;
        case 'n':
        case 'N':
            result = false;
            break;
        default:
            cout << "I'll take that for a no.\n";
            return false;
    }
}
```

Working with unknown number of data

- Input loop:

```
while (cin >> x) { /* ... */ }
```

```
cin >> x; // return cin
```

- attempt to read from `cin` (input stream) until hit the end of file or read invalid object for the type of `x`.

```
if (cin >> x) { /* ... */ }
```

```
cin >> x;  
if (cin) { /* ... */ }
```

- test the condition and read a value as a side effect.
- End-of-file signal:
 - MS Windows operating system : *Control* key + z
 - Unix and Linux operating systems: *Control* key + d
- Read unknown number of characters and write them.

```
#include <iostream>  
int main()  
{  
    char x;  
    while (std::cin >> x)  
        std::cout << x << '\n';  
    return 0;  
}
```

Program



Echo Numbers

Prog.
v

Simple computation, simple programs

- Read some “integers” and compute 1. number of inputs 2. sum and 3. average of numbers.



```
#include <iostream>

int main()
{
    using namespace std;
    int n;
    int count = 0;
    int sum = 0;
    int average;
    while (cin >> n) { // standard input loop
        ++count;
        sum += n;
    }

    average = sum / count; // integer division
    cout << count << '\t' << sum << '\t' << average << '\n';

    return 0;
}
```

Simple computation, simple programs



- Prime numbers. Read some “integers” and determine which are prime numbers. The numbers should be greater than equal 1. A positive integer p is called *prime*, if it has just two divisors, namely 1 and p . By convention 1 is not prime.

```
#include <iostream>

int main()
{
    using namespace std;

    for (;;) {
        cout << "X = (0 for exit) ";
        int x;
        if (cin >> x) {
            if (x <= 0) {
                cout << "Invalid number!";
                break;
            }
            else if (x == 1)
                cout << x << " isn't prime." << endl;
            else { // x >= 2
                bool is_prime = true;
                // inefficient and naïve implementation
                for (int i = 2; (i <= x / 2) && (is_prime = (x % i != 0)); ++i) ;
            }
        }
    }
}
```

// to be continued on next page

Forever loop, logical operators, ...

- Logical operators:

- &&

expr1 && expr2: if both expressions are true, it returns true, otherwise it returns false.

- ||

expr1 || expr2: if both expressions are false, it returns false, otherwise it returns true.

- Short-circuit evaluation*

- Logical operators: && (logical and), and || (logical or) guarantee that their left-hand operand is evaluated before their right-hand operand.

```
// continued from last page
    if (is_prime)
        cout << x << " is prime." << endl;
    else
        cout << x << " is composite." << endl;
    }
}
return 0;
}
```

```
if (i < 0 || i >= max) // ...
```

```
if (i >= 0 && i < max) // ...
```

- “infinite” loop

```
for (;;) // ...
```

```
while (true) // ...
```

Operator	Function	Use
&&	Logical and	expr && expr
	Logical or	expr expr

Simple computation, simple programs

- Read some “integers” and compute 1. number of inputs 2. sum and 3. average of numbers.



```
#include <iostream>

int main()
{
    using namespace std;
    int n;
    int count = 0;
    int sum = 0;
    int average;
    while (cin >> n) { // standard input loop
        ++count;
        sum += n;
    }

    average = sum / count; // integer division
    cout << count << '\t' << sum << '\t' << average << '\n';

    return 0;
}
```

Simple computation, simple programs



- Prime numbers. Read some “integers” and determine which are prime numbers. The numbers should be greater than equal 1. A positive integer p is called *prime*, if it has just two divisors, namely 1 and p . By convention 1 is not prime.

```
#include <iostream>

int main()
{
    using namespace std;

    for (;;) {
        cout << "X = (0 for exit) ";
        int x;
        if (cin >> x) {
            if (x <= 0) {
                cout << "Invalid number!";
                break;
            }
            else if (x == 1)
                cout << x << " isn't prime." << endl;
            else { // x >= 2
                bool is_prime = true;
                // inefficient and naïve implementation
                for (int i = 2; (i <= x / 2) && (is_prime = (x % i != 0)); ++i) ;
            }
        }
    }
}
```

// to be continued on next page

Forever loop, logical operators, ...

- Logical operators:

- &&

expr1 && expr2: if both expressions are true, it returns true, otherwise it returns false.

- ||

expr1 || expr2: if both expressions are false, it returns false, otherwise it returns true.

- Short-circuit evaluation*

- Logical operators: && (logical and), and || (logical or) guarantee that their left-hand operand is evaluated before their right-hand operand.

```
// continued from last page
    if (is_prime)
        cout << x << " is prime." << endl;
    else
        cout << x << " is composite." << endl;
    }
}
return 0;
}
```

```
if (i < 0 || i >= max) // ...
```

```
if (i >= 0 && i < max) // ...
```

- “infinite” loop

```
for (;;) // ...
```

```
while (true) // ...
```

Operator	Function	Use
&&	Logical and	expr && expr
	Logical or	expr expr

R references and further readings

[Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.

[Str08] Bjarne Stroustrup. Programming: Principles and Practice in C++, 1st edition, Addison Wesley, December 2008.

[Str13] Bjarne Stroustrup. The C++ Programming Language. 4th edition. Addison Wesley. Reading Mass. USA. May 2013.

[Str22] Bjarne Stroustrup. A Tour of C++, 3rd edition, Addison Wesley, September 2022.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

